

The eight CRC instructions are useful in calculating a cyclic redundancy checksum (CRC-32 or CRC-32C) on bytes, halfwords, words, or double words to help catch errors in large data sets (see the *Elaboration* on page 432 in Section 5.5). Similarly, the 14 Cryptographic instructions use the SIMD registers to accelerate the computation of Advanced Encryption Standard (AES) encryption, Galois/Counter Mode (GCM) encryption, and Secure Hash Algorithm (SHA) encryption.

The two move system register instructions either move data into the processor state registers from a general purpose register or an immediate (MSR) or move data from the processor state registers into a general purpose register (MRS). The 31 instructions above bring us to the halfway point of surveying the special and system instructions of ARMv8.

Starting at the top of the right column of Figure 5.48, the nine “unprivileged” loads and stores of different data widths let a processor at the EL1 interrupt level execute loads and stores that behave as if they were operating at the EL0 level, which means they can have protection faults. They act as normal loads and stores if executed at EL0 or levels higher than EL1.

The exception generation instructions include software breakpoint (BRK), halting software breakpoint (HLT), the supervisor call instructions (SVC), and two instructions similar to SVC except they go to higher exception levels: HVC goes to the hypervisor level (EL2) and SMC goes to the secure monitor level (EL3). Exception return (ERET), naturally enough, allows the program to return from an exception.

The four debugging instructions include three instructions that switch to a higher exception level (DCPS1, DCPS2, DCPS3) and one to restore to the previous processor element state (DRPS).

The six system management instructions include two general purpose ones (SYS and SYSL) and four for managing the memory hierarchy: instruction cache, data cache, address translation, and TLB invalidation.

Finally, the seven hint instructions provide a variety of architectural hints, including wait for interrupt, wait for event, send an event, yield, and NOP. These 32 instructions from the right column of Figure 5.48 bring the total to 63 special and system instructions in the full ARMv8 instruction set.

5.15

Going Faster: Cache Blocking and Matrix Multiply

Our next step in the continuing saga of improving performance of DGEMM by tailoring it to the underlying hardware is to add cache blocking to the subword parallelism and instruction level parallelism optimizations of Chapters 3 and 4. Figure 5.47 shows the blocked version of DGEMM from Figure 4.78. The changes are the same as was made earlier in going from unoptimized DGEMM in Figure 3.22 to blocked DGEMM in Figure 5.21 above. This time we take the unrolled version of DGEMM from Chapter 4 and invoke it many times on the submatrices of A, B, and C. Indeed, lines 28–34 and lines 7–8 in Figure 5.47 mirror lines 14–20 and lines 5–6 in Figure 5.21, except for incrementing the for loop in line 7 by the amount unrolled.

```

1 #include <x86intrin.h>
2 #define UNROLL (4)
3 #define BLOCKSIZE 32
4 void do_block (int n, int si, int sj, int sk,
5               double *A, double *B, double *C)
6 {
7     for ( int i = si; i < si+BLOCKSIZE; i+=UNROLL*4 )
8         for ( int j = sj; j < sj+BLOCKSIZE; j++ ) {
9             __m256d c[4];
10            for ( int x = 0; x < UNROLL; x++ )
11                c[x] = _mm256_load_pd(C+i*x*4+j*n);
12            /* c[x] = C[i][j] */
13            for( int k = sk; k < sk+BLOCKSIZE; k++ )
14                {
15                    __m256d b = _mm256_broadcast_sd(B+k+j*n);
16                    /* b = B[k][j] */
17                    for (int x = 0; x < UNROLL; x++)
18                        c[x] = _mm256_add_pd(c[x], /* c[x]+=A[i][k]*b */
19                                           _mm256_mul_pd(_mm256_load_pd(A+n*k+x*4+i), b));
20                }
21
22
23            for ( int x = 0; x < UNROLL; x++ )
24                _mm256_store_pd(C+i*x*4+j*n, c[x]);
25            /* C[i][j] = c[x] */
26        }
27
28 void dgemm (int n, double* A, double* B, double* C)
29 {
30     for ( int sj = 0; sj < n; sj += BLOCKSIZE )
31         for ( int si = 0; si < n; si += BLOCKSIZE )
32             for ( int sk = 0; sk < n; sk += BLOCKSIZE )
33                 do_block(n, si, sj, sk, A, B, C);
34 }

```

FIGURE 5.47 Optimized C version of DGEMM from Figure 4.78 using cache blocking. These changes are the same ones found in Figure 5.21. The assembly language produced by the compiler for the `do_block` function is nearly identical to Figure 4.79. Once again, there is no overhead to call the `do_block` because the compiler inlines the function call.

Unlike the earlier chapters, we do not show the resulting x86 code because the inner loop code is nearly identical to Figure 4.79, as the blocking does not affect the computation, just the order that it accesses data in memory. What does change is the bookkeeping integer instructions to implement the loops. It expands from 14 instructions before the inner loop and eight after the loop for Figure 4.78 to 40 and 28 instructions respectively for the bookkeeping code generated

Type	Mnemonic	Instruction	Type	Mnemonic	Instruction
Non-cache	LDNP	Load Non-temporal Pair	Unprivileged	LDTR	Load Unprivileged register
	STNP	Store Non-temporal Pair		LDTRB	Load Unprivileged byte
Barrier	CLREX	Clear exclusive monitor		LDTRSB	Load Unprivileged signed byte
	DSB	Data synchronization barrier		LDTRH	Load Unprivileged halfword
	DMB	Data memory barrier		LDTRSH	Load Unprivileged signed halfword
	ISB	Instruction synchronization barrier		LDTRSW	Load Unprivileged signed word
CRC	CRC32B	CRC-32 sum from byte		STTR	Store Unprivileged register
	CRC32H	CRC-32 sum from halfword		STTRB	Store Unprivileged byte
	CRC32W	CRC-32 sum from word		STTRH	Store Unprivileged halfword
	CRC32X	CRC-32 sum from doubleword	Exception	BRK	Software breakpoint instruction
	CRC32CB	CRC-32C sum from byte		HLT	Halting software breakpoint instruction
	CRC32CH	CRC-32C sum from halfword		HVC	Generate exception targeting Exception level 2
	CRC32CW	CRC-32C sum from word		SMC	Generate exception targeting Exception level 3
	CRC32CX	CRC-32C sum from doubleword		SVC	Generate exception targeting Exception level 1
Crypto	AESD	AES single round decryption	Debug	ERET	Exception return using current ELR and SPSR
	AESE	AES single round encryption		DCPS1	Debug switch to Exception level 1
	AESIMC	AES inverse mix columns		DCPS2	Debug switch to Exception level 2
	AESMC	AES mix columns		DCPS3	Debug switch to Exception level 3
	PMULL	Polynomial multiply long		DRPS	Debug restore PE state
	SHA1C	SHA1 hash update (choose)	System	SYS	System instruction
	SHA1H	SHA1 fixed rotate		SYSL	System instruction with result
	SHA1M	SHA1 hash update (majority)		IC	Instruction cache maintenance
	SHA1P	SHA1 hash update (parity)		DC	Data cache maintenance
	SHA1SU0	SHA1 schedule update 0		AT	Address translation
	SHA1SU1	SHA1 schedule update 1	Hint	TLBI	TLB Invalidate
	SHA256H	SHA256 hash update (part 1)		NOP	No operation
	SHA256H2	SHA256 hash update (part 2)		YIELD	Yield hint
	SHA256SU0	SHA256 schedule update 0		WFE	Wait for event
	SHA256SU1	SHA256 schedule update 1		WFI	Wait for interrupt
Sys Reg	MRS	Move system register to general-purpose register		SEV	Send event
	MSR	Move general-purpose register or immediate to system register		SEVL	Send event local
				HINT	Unallocated hint

FIGURE 5.48 The list of assembly language instructions for the systems and special operations in the full ARMv8 instruction set. Bold means the instruction is also in LEGv8.

for Figure 5.47. Nevertheless, the extra instructions executed pale in comparison to the performance improvement of reducing cache misses. Figure 5.49 compares unoptimized to optimized for subword parallelism, instruction level parallelism, and caches. Blocking improves performance over unrolled AVX code by factors of 2 to 2.5 for the larger matrices. When we compare unoptimized code to the code with all three optimizations, the performance improvement is factors of 8 to 15, with the largest increase for the largest matrix.

Elaboration: As mentioned in the *Elaboration* in Section 3.9, these results are with Turbo mode turned off. As in Chapters 3 and 4, when we turn it on, we improve all the results by the temporary increase in the clock rate of $3.3/2.6 = 1.27$. Turbo mode works particularly well in this case because it is using only a single core of an eight-core chip. However, if we want to run fast we should use all cores, which we'll see in Chapter 6.

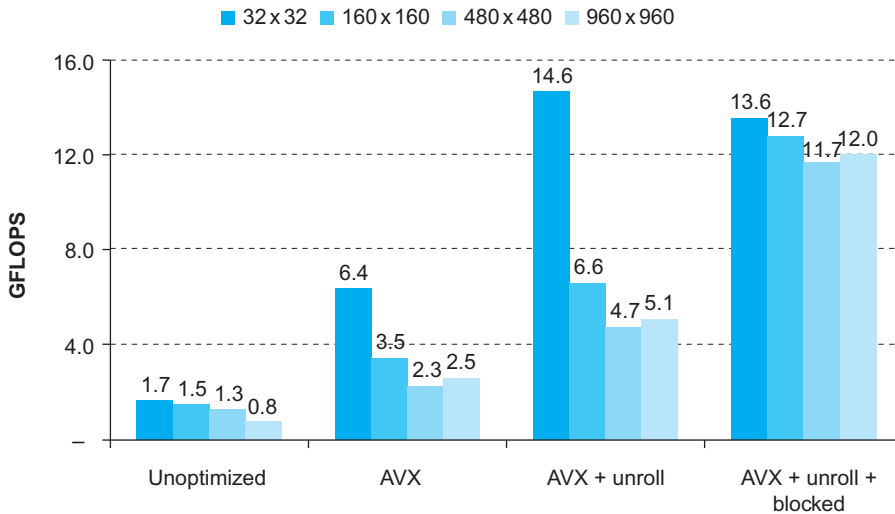


FIGURE 5.49 Performance of four versions of DGEMM from matrix dimensions 32×32 to 960×960 . The fully optimized code for the largest matrix is almost 15 times as fast the unoptimized version in [Figure 3.22 in Chapter 3](#).

5.16 Fallacies and Pitfalls

As one of the most naturally quantitative aspects of computer architecture, the memory hierarchy would seem to be less vulnerable to fallacies and pitfalls. Not only have there been many fallacies propagated and pitfalls encountered, but some have led to major negative outcomes. We start with a pitfall that often traps students in exercises and exams.

Pitfall: Ignoring memory system behavior when writing programs or when generating code in a compiler.

This could be rewritten as a fallacy: “Programmers can ignore memory hierarchies in writing code.” The evaluation of sort in [Figure 5.19](#) and of cache blocking in Section 5.14 demonstrate that programmers can easily double performance if they factor the behavior of the memory system into the design of their algorithms.

Pitfall: Forgetting to account for byte addressing or the cache block size in simulating a cache.

When simulating a cache (by hand or by computer), we need to make sure we account for the effect of byte addressing and multiword blocks in determining into which cache block a given address maps. For example, if we have a 32-byte direct-mapped cache with a block size of 4 bytes, the byte address 36 maps into block 1

of the cache, since byte address 36 is block address 9 and $(9 \bmod 8) = 1$. On the other hand, if address 36 is a word address, then it maps into block $(36 \bmod 8) = 4$. Make sure the problem clearly states the base of the address.

In like fashion, we must account for the block size. Suppose we have a cache with 256 bytes and a block size of 32 bytes. Into which block does the byte address 300 fall? If we break the address 300 into fields, we can see the answer:

63	62	61	...	...	...	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	...	...	...	0	0	0	1	0	0	1	0	1	1	0	0
										Cache block number			Block offset				

Block address

Byte address 300 is block address

$$\left\lfloor \frac{300}{32} \right\rfloor = 9$$

The number of blocks in the cache is

$$\left\lfloor \frac{256}{32} \right\rfloor = 8$$

Block number 9 falls into cache block number $(9 \bmod 8) = 1$.

This mistake catches many people, including the authors (in earlier drafts) and instructors who forget whether they intended the addresses to be in doublewords, words, bytes, or block numbers. Remember this pitfall when you tackle the exercises.

Pitfall: Having less set associativity for a shared cache than the number of cores or threads sharing that cache.



Without extra care, a **parallel** program running on 2^n processors or threads can easily allocate data structures to addresses that would map to the same set of a shared L2 cache. If the cache is at least 2^n -way associative, then these accidental conflicts are hidden by the hardware from the program. If not, programmers could face apparently mysterious performance bugs—actually due to L2 conflict misses—when migrating from, say, a 16-core design to 32-core design if both use 16-way associative L2 caches.

Pitfall: Using average memory access time to evaluate the memory hierarchy of an out-of-order processor.

If a processor stalls during a cache miss, then you can separately calculate the memory-stall time and the processor execution time, and hence evaluate the memory hierarchy independently using average memory access time (see page 413).

If the processor continues to execute instructions, and may even sustain more cache misses during a cache miss, then the only accurate assessment of the memory hierarchy is to simulate the out-of-order processor along with the memory hierarchy.

Pitfall: Extending an address space by adding segments on top of an unsegmented address space.

During the 1970s, many programs grew so large that not all the code and data could be addressed with just a 16-bit address. Computers were then revised to offer 32-bit addresses, either through an unsegmented 32-bit address space (also called a *flat address space*) or by adding 16 bits of segment to the existing 16-bit address. From a marketing point of view, adding segments that were programmer-visible and that forced the programmer and compiler to decompose programs into segments could solve the addressing problem. Unfortunately, there is trouble any time a programming language wants an address that is larger than one segment, such as indices for large arrays, unrestricted pointers, or reference parameters. Moreover, adding segments can turn every address into two words—one for the segment number and one for the segment offset—causing problems in the use of addresses in registers.

Fallacy: Disk failure rates in the field match their specifications.

Two recent studies evaluated large collections of disks to check the relationship between results in the field compared to specifications. One study was of almost 100,000 disks that had quoted MTTF of 1,000,000 to 1,500,000 hours, or AFR of 0.6% to 0.8%. They found AFRs of 2% to 4% to be common, often three to five times higher than the specified rates [Schroeder and Gibson, 2007]. A second study of more than 100,000 disks at Google, which had a quoted AFR of about 1.5%, saw failure rates of 1.7% for drives in their first year rise to 8.6% for drives in their third year, or about five to six times the declared rate [Pinheiro, Weber, and Barroso, 2007].

Fallacy: Operating systems are the best place to schedule disk accesses.

As mentioned in Section 5.2, higher-level disk interfaces offer logical block addresses to the host operating system. Given this high-level abstraction, the best an OS can do to try to help performance is to sort the logical block addresses into increasing order. However, since the disk knows the actual mapping of the logical addresses onto the physical geometry of sectors, tracks, and surfaces, it can reduce the rotational and seek latencies by rescheduling.

For example, suppose the workload is four reads [Anderson, 2003]:

Operation	Starting LBA	Length
Read	724	8
Read	100	16
Read	9987	1
Read	26	128

The host might reorder the four reads into logical block order:

Operation	Starting LBA	Length
Read	26	128
Read	100	16
Read	724	8
Read	9987	1

Depending on the relative location of the data on the disk, reordering could make it worse, as Figure 5.50 shows. The disk-scheduled reads complete in three-quarters of a disk revolution, but the OS-scheduled reads take three revolutions.

Pitfall: Implementing a virtual machine monitor on an instruction set architecture that wasn't designed to be virtualizable.

Many architects in the 1970s and 1980s weren't careful to make sure that all instructions reading or writing information related to hardware resource information were privileged. This *laissez-faire* attitude causes problems for VMMs for all of these architectures, including the x86, which we use here as an example.

Figure 5.51 describes the 18 instructions that cause problems for virtualization [Robin and Irvine, 2000]. The two broad classes are instructions that

- Read control registers in user mode that reveals that the guest operating system is running in a virtual machine (such as POPF, mentioned earlier)
- Check protection as required by the segmented architecture but assume that the operating system is running at the highest privilege level

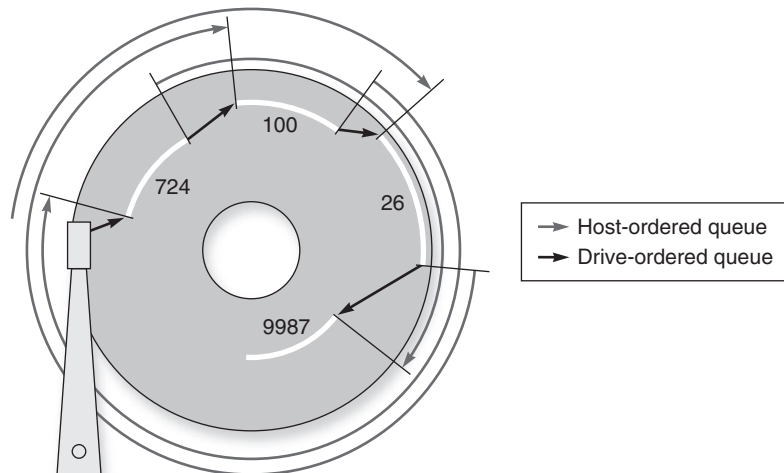


FIGURE 5.50 Example showing OS versus disk schedule accesses, labeled host-ordered versus drive-ordered. The former takes three revolutions to complete the four reads, while the latter completes them in just three-fourths of a revolution. From Anderson [2003].

Problem category	Problem x86 instructions
Access sensitive registers without trapping when running in user mode	Store global descriptor table register (SGDT) Store local descriptor table register (SLDT) Store interrupt descriptor table register (SIDT) Store machine status word (SMSW) Push flags (PUSHF, PUSHFD) Pop flags (POPF, POPFD)
When accessing virtual memory mechanisms in user mode, instructions fail the x86 protection checks	Load access rights from segment descriptor (LAR) Load segment limit from segment descriptor (LSL) Verify if segment descriptor is readable (VERR) Verify if segment descriptor is writable (VERW) Pop to segment register (POP CS, POP SS, . . .) Push segment register (PUSH CS, PUSH SS, . . .) Far call to different privilege level (CALL) Far return to different privilege level (RET) Far jump to different privilege level (JMP) Software interrupt (INT) Store segment selector register (STR) Move to/from segment registers (MOVE)

FIGURE 5.51 Summary of 18 x86 instructions that cause problems for virtualization [Robin and Irvine, 2000]. The first five instructions in the top group allow a program in user mode to read a control register, such as descriptor table registers, without causing a trap. The pop flags instruction modifies a control register with sensitive information but fails silently when in user mode. The protection checking of the segmented architecture of the x86 is the downfall of the bottom group, as each of these instructions checks the privilege level implicitly as part of instruction execution when reading a control register. The checking assumes that the OS must be at the highest privilege level, which is not the case for guest VMs. Only the Move to segment register tries to modify control state, and protection checking foils it as well.

To simplify implementations of VMMs on the x86, both AMD and Intel have proposed extensions to the architecture via a new mode. Intel's VT-x provides a new execution mode for running VMs, an architected definition of the VM state, instructions to swap VMs rapidly, and a large set of parameters to select the circumstances where a VMM must be invoked. Altogether, VT-x adds 11 new instructions for the x86. AMD's Pacifica makes similar proposals.

An alternative to modifying the hardware is to make small changes to the operating system to avoid using the troublesome pieces of the architecture. This technique is called *paravirtualization*, and the open source Xen VMM is a good example. The Xen VMM provides a guest OS with a virtual machine abstraction that uses only the easy-to-virtualize parts of the physical x86 hardware on which the VMM runs.

5.17 Concluding Remarks



H I E R A R C H Y



P R E D I C T I O N

prefetching A technique in which data blocks needed in the future are brought into the cache early by using special instructions that specify the address of the block.

The difficulty of building a memory system to keep pace with faster processors is underscored by the fact that the raw material for main memory, DRAMs, is essentially the same in the fastest computers as it is in the slowest and cheapest.

It is the principle of locality that gives us a chance to overcome the long latency of memory access—and the soundness of this strategy is demonstrated at all levels of the **memory hierarchy**. Although these levels of the hierarchy look quite different in quantitative terms, they follow similar strategies in their operation and exploit the same properties of locality.

Multilevel caches make it possible to use more cache optimizations more easily for two reasons. First, the design parameters of a lower-level cache are different from a first-level cache. For example, because a lower-level cache will be much larger, it is possible to use bigger block sizes. Second, a lower-level cache is not constantly being used by the processor, as a first-level cache is. This allows us to consider having the lower-level cache do something when it is idle that may be useful in preventing future misses.

Another trend is to seek software help. Efficiently managing the memory hierarchy using a variety of program transformations and hardware facilities is a major focus of compiler enhancements. Two different ideas are being explored. One idea is to reorganize the program to enhance its spatial and temporal locality. This approach focuses on loop-oriented programs that use sizable arrays as the major data structure; large linear algebra problems are a typical example, such as DGEMM. By restructuring the loops that access the arrays, substantially improved locality—and, therefore, cache performance—can be obtained.

Another approach is **prefetching**. In prefetching, a block of data is brought into the cache before it is actually referenced. Many microprocessors use hardware prefetching to try to *predict* accesses that may be difficult for software to notice.

A third approach is special cache-aware instructions that optimize memory transfer. For example, the microprocessors in [Section 6.10 in Chapter 6](#) use an optimization that does not fetch the contents of a block from memory on a write miss because the program is going to write the full block. This optimization significantly reduces memory traffic for one kernel.

As we will see in [Chapter 6](#), memory systems are a central design issue for parallel processors. The growing significance of the memory hierarchy in determining system performance means that this important area will continue to be a focus for both designers and researchers for some years to come.



Historical Perspective and Further Reading

This section, which appears online, gives an overview of memory technologies, from mercury delay lines to DRAM, the invention of the memory hierarchy, protection mechanisms, and virtual machines, and concludes with a brief history of operating systems, including CTSS, MULTICS, UNIX, BSD UNIX, MS-DOS, Windows, and Linux.

5.19

Exercises

Assume memory is byte addressable and words are 64 bits, unless specified otherwise.

5.1 In this exercise we look at memory locality properties of matrix computation. The following code is written in C, where elements within the same row are stored contiguously. Assume each word is a 64-bit integer.

```
for (I=0; I<8; I++)
    for (J=0; J<8000; J++)
        A[I][J]=B[I][0]+A[J][I];
```

5.1.1 [5] <\$5.1> How many 64-bit integers can be stored in a 16-byte cache block?

5.1.2 [5] <\$5.1> Which variable references exhibit temporal locality?

5.1.3 [5] <\$5.1> Which variable references exhibit spatial locality?

Locality is affected by both the reference order and data layout. The same computation can also be written below in Matlab, which differs from C in that it stores matrix elements within the same column contiguously in memory.

```
for I=1:8
    for J=1:8000
        A(I,J)=B(I,0)+A(J,I);
    end
end
```

5.1.4 [5] <\$5.1> Which variable references exhibit temporal locality?

5.1.5 [5] <\$5.1> Which variable references exhibit spatial locality?



Historical Perspective and Further Reading

This history section gives an overview of memory technologies, from mercury delay lines to DRAM, the invention of the memory hierarchy, and protection mechanisms, and concludes with a brief history of operating systems, including CTSS, MULTICS, UNIX, BSD UNIX, MS-DOS, Windows, and Linux.

The developments of most of the concepts in this chapter have been driven by revolutionary advances in the technology we use for memory. Before we discuss how memory hierarchies were evolved, let's take a brief tour of the development of memory technology.

The ENIAC had only a small number of registers (about 20) for its storage and implemented these with the same basic vacuum tube technology that it used for building logic circuitry. However, the vacuum tube technology was far too expensive to be used to build a larger memory capacity. Eckert came up with the idea of developing a new technology based on mercury delay lines. In this technology, electrical signals were converted into vibrations that were sent down a tube of mercury, reaching the other end, where they were read out and recirculated. One mercury delay line could store about 0.5 Kbits. Although these bits were accessed serially, the mercury delay line was about a hundred times more cost-effective than vacuum tube memory. The first known working mercury delay lines were developed at Cambridge for the EDSAC. [Figure e5.17.1](#) shows the mercury delay lines of the EDSAC, which had 32 tanks and 512 36-bit words.

Despite the tremendous advance offered by the mercury delay lines, they were terribly unreliable and still rather expensive. The breakthrough came with the invention of core memory by J. Forrester at MIT as part of the Whirlwind project in the early 1950s (see [Figure e5.17.2](#)). Core memory uses a ferrite core, which can be magnetized, and once magnetized, it acts as a store (just as a magnetic recording tape stores information). A set of wires running through the center of the core, which had a dimension of 0.1–1.0 millimeters, makes it possible to read the value stored on any ferrite core. The Whirlwind eventually included a core memory with 2048 16-bit words, or 32 Kbits. Core memory was a tremendous advance: it was cheaper, faster, considerably more reliable, and had higher density. Core memory was so much better than the alternatives that it became the dominant memory technology only a few years after its invention and remained so for nearly 20 years.

...the one single development that put computers on their feet was the invention of a reliable form of memory, namely, the core memory.... Its cost was reasonable, it was reliable and, because it was reliable, it could in due course be made large.

Maurice Wilkes,
Memoirs of a Computer Pioneer, 1985

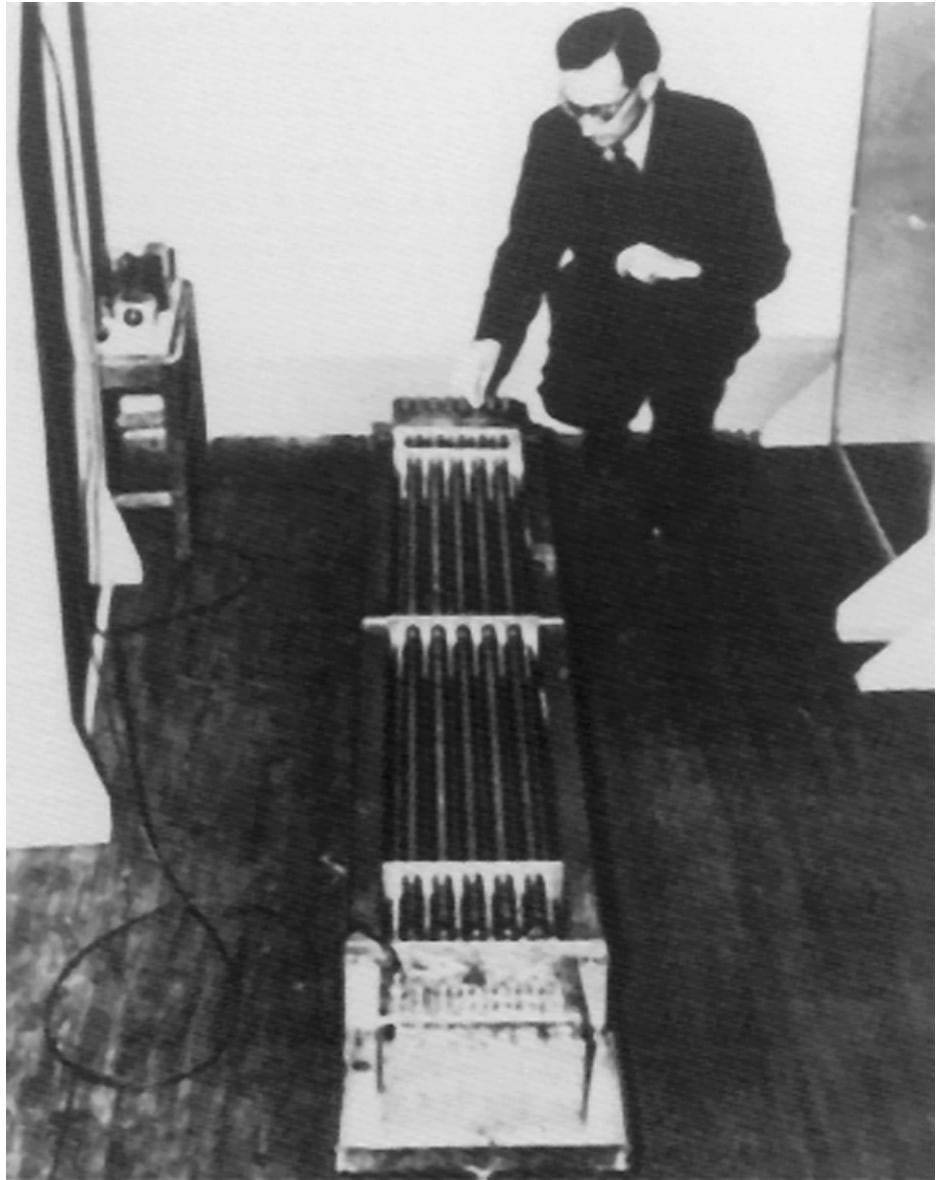


FIGURE e5.17.1 The mercury delay lines in the EDSAC. This technology made it possible to build the first stored-program computer. The young engineer in this photograph is none other than Maurice Wilkes, the lead architect of the EDSAC.

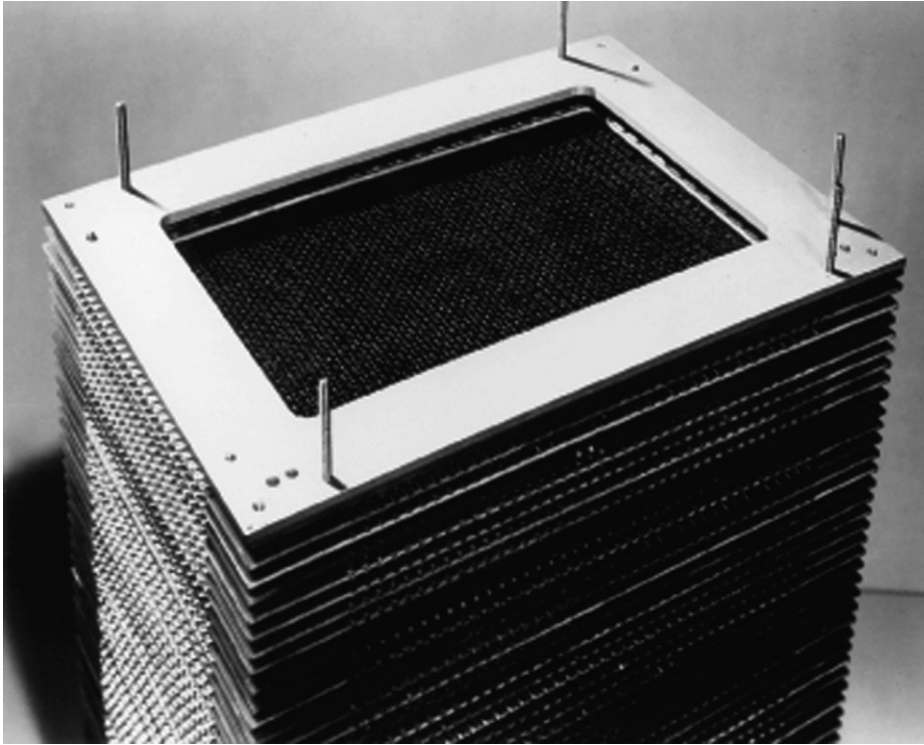


FIGURE e5.17.2 A core memory plane from the Whirlwind containing 256 cores arranged in a 16×16 array. Core memory was invented for the Whirlwind, which was used for air defense problems, and is now on display at the Smithsonian. (Incidentally, Ken Olsen, the founder of Digital and its president for 20 years, built the computer that tested these core memories; it was his first computer.)

The technology that replaced core memory was the same one that we now use both for logic and for memory: the integrated circuit. While registers were built out of transistorized memory in the 1960s, and IBM computers used transistorized memory for microcode store and caches in 1970, building main memory out of transistors remained prohibitively expensive until the development of the integrated circuit. With the integrated circuit, it became possible to build a DRAM (dynamic random access memory—see Appendix A for a description). The first DRAMs were built at Intel in 1970, and the computers using DRAM memories (as a high-speed option to core) came shortly thereafter; they used 1 Kbit DRAMs. In fact, computer folklore says that Intel developed the microprocessor partly to help

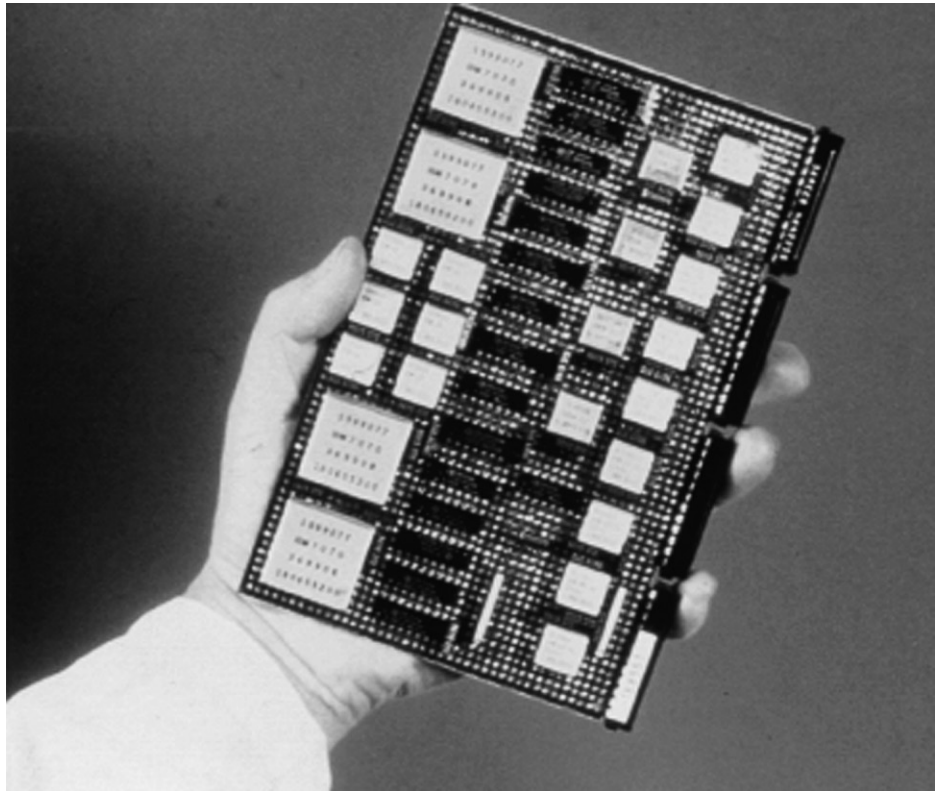


FIGURE e5.17.3 An early DRAM board. This board uses 18 Kbit chips.

sell more DRAM. [Figure e5.17.3](#) shows an early DRAM board. By the late 1970s, core memory had become a historical curiosity. Just as core memory technology had allowed a tremendous expansion in memory size, DRAM technology allowed a comparable expansion. In the 1990s, many personal computers had as much memory as the largest computers using core memory ever had.

Nowadays, DRAMs are typically packaged with multiple chips on a little board called a DIMM (dual inline memory module). The SIMM (single inline memory module) shown in [Figure e5.17.4](#) contains a total of 1 MB and sold for about \$5 in 1997. As of 2004, DIMMs were available with up to 1024 MB and sold for about \$100. While DRAMs will remain the dominant memory technology for some time to come, innovations in the packaging of DRAMs to provide both higher bandwidth and greater density are ongoing.

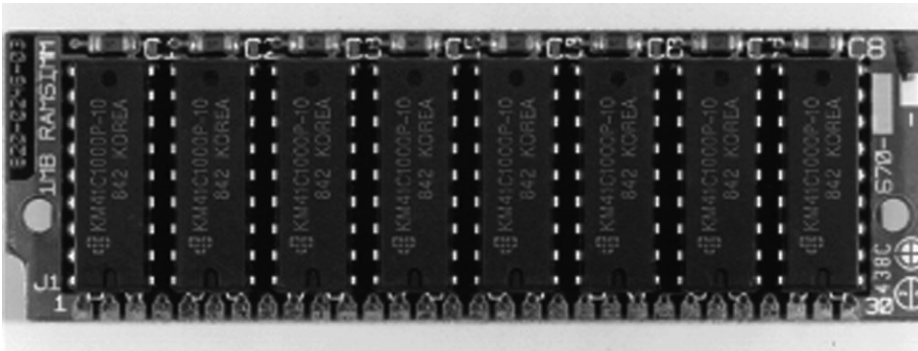


FIGURE e5.17.4 A 1 MB SIMM, built in 1986, using 1 Mbit chips. This SIMM sold for about \$5/MB in 1997. As of 2006, most main memory is packed in DIMMs similar to this, though using much higher-density memory chips (1 Gbit).

The Development of Memory Hierarchies

Although the pioneers of computing foresaw the need for a **memory hierarchy** and coined the term, the automatic management of two levels was first proposed by Kilburn and his colleagues and demonstrated at the University of Manchester with the Atlas computer, which implemented virtual memory. This was the year *before* the IBM 360 was announced. IBM planned to include virtual memory with the next generation (System/370), but the OS/360 operating system wasn't up to the challenge in 1970. Virtual memory was announced for the 370 family in 1972, and it was for this computer that the term *translation-lookaside buffer* was coined. All but some embedded computers use virtual memory today.

The problems of inadequate address space have plagued designers repeatedly. The architects of the PDP-11 identified a small address space as the only architectural mistake from which it is difficult to recover. When the PDP-11 was designed, core memory densities were increasing at a very slow rate, and the competition from 100 other minicomputer companies meant that DEC might not have a cost-competitive product if every address had to go through the 16-bit datapath twice—hence, the decision to add just 4 more address bits than the predecessor of the PDP-11, to 16 from 12. The architects of the IBM 360 were aware of the importance of address size and planned for the architecture to extend to 32 bits of address. Only 24 bits were used in the IBM 360, however, because the low-end 360 models would have been even slower with the larger addresses. Unfortunately, the expansion effort was greatly complicated by programmers who stored extra information in the upper 8



“unused” address bits. The wider address lasted until 2000, when IBM expanded the architecture to 64 bits in the z-series.

Running out of address space has often been the cause of death for an architecture, while other architectures have managed to make the transition to a larger address space. For example, the PDP-11, a 16-bit computer, was replaced by the 32-bit VAX. The 80386 extended the 80286 architecture from a segmented 24-bit address space to a flat 32-bit address space in 1985. In the 1990s, several RISC instruction sets made the transition from 32-bit addressing to 64-bit addressing by providing a compatible extension of their instruction sets. MIPS was the first to do so. A decade later, Intel and HP announced the IA-64 in large part to provide a 64-bit address successor to the 32-bit Intel IA-32 and HP Precision architectures. The evolutionary AMD64 won that battle versus the revolutionary IA-64, and all but a few thousand of the 64-bit address computers from Intel are based on the x86.

Many of the early ideas in memory hierarchies originated in England. Just a few years after the Atlas paper, [Wilkes \[1965\]](#) published the first paper describing the concept of a cache, calling it a “slave”:

The use is discussed of a fast core memory of, say, 32,000 words as slave to a slower core memory of, say, one million words in such a way that in practical cases the effective access time is nearer that of the fast memory than that of the slow memory.

This two-page paper describes a direct-mapped cache. Although this was the first publication on caches, the first implementation was probably a direct-mapped instruction cache built at the University of Cambridge by Scarrott and described at the 1965 IFIP Congress. It was based on tunnel diode memory, the fastest form of memory available at the time.

Subsequent to that publication, IBM started a project that led to the first commercial computer with a cache, the IBM 360/85. Gibson at IBM recognized that memory-accessing behavior would have a significant impact on performance. He described how to measure program behavior and cache behavior and showed that the miss rate varies between programs. Using a sample of 20 programs (each with 3 million references—an incredible number for that time), Gibson analyzed the effectiveness of caches using average memory access time as the metric. Conti, Gibson, and Pitowsky described the resulting performance of the 360/85 in the first paper to use the term *cache* in 1968. Since this early work, it has become clear that caches are one of the most important ideas not only in computer architecture but in software systems as well. The idea of caching has found applications in operating systems, networking systems, databases, and compilers, to name a few. There are

thousands of papers on the topic of caching, and it continues to be a popular area of research.

One of the first papers on nonblocking caches was by Kroft in 1981, who may have coined the term. He later explained that he was the first to design a computer with a cache at Control Data Corporation, and when using old concepts for new mechanisms, he hit upon the idea of allowing his two-ported cache to continue to service other accesses on a miss.

Multilevel caches were the inevitable resolution to the lack of improvement in main memory latency and the higher clock rates of microprocessors. Only those in the field for a while are surprised by the size of some second- or third-level caches, as they are larger than main memories of past machines. The other surprise is that the number of levels is continually increasing, even on a single-chip microprocessor.

Disk Storage

In 1956, IBM developed the first disk storage system with both moving heads and multiple disk surfaces in San Jose, helping to seed the birth of the magnetic storage industry in the southern end of Silicon Valley. Reynold B. Johnson led the development of the IBM 305 RAMAC (Random Access Method of Accounting and Control). It could store 5 million characters (5 MB) of data on 50 disks, each 24 inches in diameter. The RAMAC is shown in [Figures e5.17.5 and e5.17.6](#). Although the disk pioneers would be amazed at the size, cost, and capacity of modern disks, the basic mechanical design is the same as the RAMAC.

Moving-head disks quickly became the dominant high-speed magnetic storage, though their high cost meant that magnetic tape continued to be used extensively until the 1970s. The next key milestone for hard disks was the removable hard disk drive developed by IBM in 1962; this made it possible to share the expensive drive electronics and helped disks overtake tapes as the preferred storage medium. [Figure e5.17.7](#) shows a removable disk drive and the multiplatter disk used in the drive. IBM also invented the floppy disk drive in 1970, originally to hold microcode for the IBM 370 series. Floppy disks became popular with the PC about 10 years later.

The sealed Winchester disk, which was developed by IBM in 1973, completely dominates disk technology today. Winchester disks benefited from two related properties. First, reductions in the cost of the disk electronics made it unnecessary to share the electronics and thus made nonremovable disks economical. Since the disk was fixed and could be in a sealed enclosure, both the environmental and control problems were greatly reduced, allowing significant gains in density. The first disk that IBM shipped had two spindles, each with a 30 MB disk; the moniker “30-30” for the disk led to the name Winchester. Winchester disks grew rapidly in popularity in the 1980s, completely replacing removable disks by the middle of that decade.

The historic role of IBM in the disk industry came to an end in 2002, when IBM sold its disk storage division to Hitachi. IBM continues to make storage subsystems, but it purchases its disk drives from others.

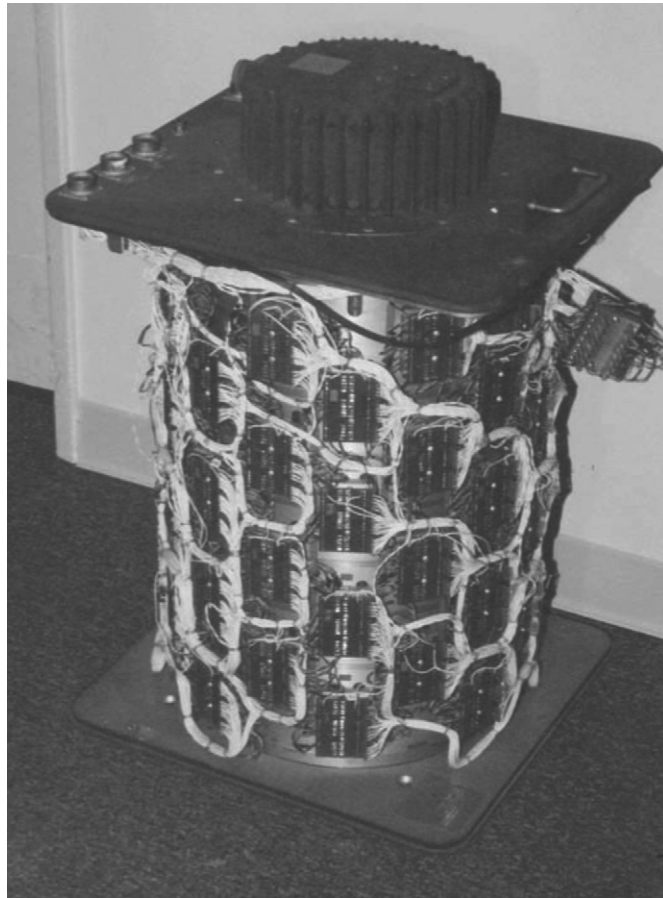


FIGURE e5.17.5 A magnetic drum made by Digital Development Corporation in the 1960s and used on a CDC machine. The electronics supporting the read/write heads can be seen on the outside of the drum.

A Very Brief History of Flash Memory

Flash memory was invented by researchers at Toshiba in the 1980s. They invented both the NOR-based Flash memory in 1984 and the denser NAND-based Flash memory in 1989. The first use was in digital cameras, starting with the CompactFlash form factor for NOR Flash memory and the SmartMedia form factor for NAND Flash memory. Today, all digital cameras, cell phones, music players, and tablets rely on Flash memory, and an increasing fraction of laptops use flash memory instead of disk.



FIGURE e5.17.6 The RAMAC disk drive from IBM, made in 1956, was the first disk drive with a moving head and the first with multiple platters. The IBM storage technology Web site has a discussion of IBM's major contributions to storage technology.

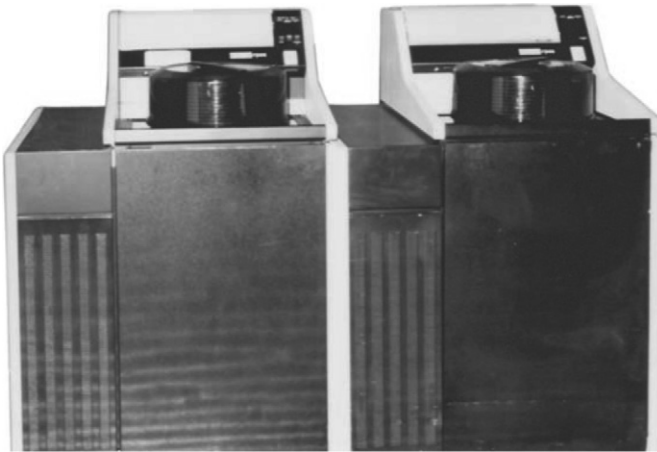


FIGURE e5.17.7 This is a DEC disk drive and the removable pack. These disks became popular starting in the mid-1960s and dominated disk technology until Winchester drives in the late 1970s. This drive was made in the mid-1970s; each disk pack in this drive could hold 80 MB.

A Brief History of Databases

Although there had been data stores of punch cards and later magnetic tapes, the emergence of the magnetic disk led to modern databases.

In 1961, Charles Bachman at General Electric created a pioneering database management system called Integrated Data Store (IDS) to take advantage of the new magnetic disks. In 1971, Bachman and others published standards on how to manage databases using Cobol programs, named the Codasyl approach after

the standards committee on which they served. Many companies offered Codasyl-compatible databases, but not IBM. IBM had introduced IMS in 1968, which was derived from IBM's work on the NASA Apollo project. Both Codasyl databases and IMS are classified as navigational databases because programs had to navigate through the data.

Ted Codd, a researcher at IBM, thought the navigational approach was wrong-headed. He recalled that people didn't write programs when dealing with the old punch card databases. Instead, they set up data flows through series of punch card machines that would perform simple functions like copy or sort. Once the card machines were set up, you just pushed all the cards through to get your results. In his view, users should only declare the type of data they were looking for and leave it up to computers to process it. In 1970, he published a new way to organize and access data called the relational model. It was based on set theory; data were independent of the implementation and users described what they were looking for in a declarative, nonprocedural language.

This paper led to considerable controversy within IBM, because it already had a database product. Codd even arranged a public debate between him and Bachman, which led to internal criticism at IBM that Codd was undermining IMS. The good news was that the debate led researchers at IBM and U.C. Berkeley to try to demonstrate the viability of relational databases by building System R and Ingres.

System R in 1974–79 demonstrated its feasibility and, perhaps more importantly, created the Structured Query Language (SQL) that is still widely used today. However, these results were not sufficient to convince IBM, and some of the researchers left IBM to build relational databases for other companies.

Mike Stonebraker and Gene Wong were interested in geographic data systems, and in 1973 they decided to pursue relational databases. Rather than build on IBM mainframes, the Ingres project was built on DEC minicomputers and Unix. Ingres was important because it led to a company that tried to commercialize the ideas, because 1000 copies of its source code were openly distributed, and because it trained a generation of database developers and researchers. The code and people led to many other companies, including Sybase. Larry Ellison started Oracle by first reading the papers from the System R and Ingres groups and then by hiring people who worked on those projects. Microsoft later purchased a copy of Sybase sources that became the foundation of its SQL Server product.

Relational databases matured in the 1980s, with IBM developing its own relational databases, including DB2. The 1990s saw both the development of object-oriented databases, to address the impedance mismatch between databases and programming, and the evolution of parallel databases for analytic processing and data mining.

ACM showered awards on this community. The ACM Turing Award went to Charles Bachman in 1973 for his contributions via IDS and the Codasyl group. Codd won it in 1980 for the relational model. In 1988, the developers of System R (Donald Chamberlin, Jim Gray, Raymond Lorie, Gianfranco Putzolu, Patricia Selinger, and Irving Traiger) shared the ACM Systems Software Award with the developers of Ingres (Gerald Held, Michael Stonebraker, and Eugene Wong). Jim

Gray won the Turing Award in 1998 for his contributions to transaction processing and databases. Stonebraker won it in 2014 for contributions to the concepts and practices underlying modern database systems. Finally, the first two ACM SIGMOD Innovations Awards went to Stonebraker and Gray, and the 2002 and 2003 editions went to Selinger and Chamberlin.

RAID

The small-form-factor hard disks for PCs in the mid-1980s led a group at Berkeley to propose redundant arrays of inexpensive disks (RAID). This group had worked on the reduced instruction set computer effort and so expected much faster processors to become available. Their two questions were: What could be done with the small disks that accompanied their PCs? What could be done in the area of I/O to keep up with much faster processors? They argued to replace one large mainframe drive with 50 small drives, as you could get much greater performance with that many independent arms. The many small drives even offered savings in power consumption and floor space.

The downside of many disks was much lower MTTF. Hence, on their own they reasoned out the advantages of redundant disks and rotating parity to address how to get greater performance with many small drives yet have reliability as high as that of a single mainframe disk.

The problem they experienced when explaining their ideas was that some researchers had heard of disk arrays with some form of redundancy, and they didn't understand the Berkeley proposal. Hence, the first RAID paper [Patterson, Gibson, and Katz 1987] is not only a case for arrays of small-form-factor disk drives, but also something of a tutorial and classification of existing work on disk arrays. Mirroring (RAID 1) had long been used in fault-tolerant computers such as those sold by Tandem. Thinking Machines had arrays with 32 data disks and seven check disks using ECC for correction (RAID 2) in 1987, and Honeywell Bull had a RAID 2 product even earlier. Also, disk arrays with a single parity disk had been used in scientific computers in the same time frame (RAID 3). Their paper then described a single parity disk with support for sector accesses (RAID 4) and rotated parity (RAID 5). [Chen et al. \[1994\]](#) survey the original RAID ideas, commercial products, and other developments.

Unknown to the Berkeley group, engineers at IBM working on the AS/400 computer also came up with rotated parity to give greater reliability for a collection of large disks. IBM filed a patent on RAID 5 shortly before the Berkeley group submitted their paper. Patents for RAID 1, RAID 2, and RAID 3 from several companies predate the IBM RAID 5 patent, which has led to plenty of courtroom action.

EMC had been a supplier of DRAM boards for IBM computers, but around 1988 new policies from IBM made it nearly impossible for EMC to continue to sell IBM memory boards. The Berkeley paper crossed the desks of EMC executives, and so they decided to go after the market dominated by IBM disk storage products. As the paper advocated, their model was to use many small drives to compete with mainframe drives, and EMC announced a RAID product in 1990. It relied on

mirroring (RAID 1) for reliability; RAID 5 products came much later for EMC. Over the next year, Micropolis offered a RAID 3 product; Compaq offered a RAID 4 product; and Data General, IBM, and NCR offered RAID 5 products.

The RAID ideas soon spread to the rest of the workstation and server industry. An article explaining RAID in *Byte* magazine led to RAID products being offered on desktop PCs, which was something of a surprise to the Berkeley group. They had focused on performance with good availability, but higher availability was attractive to the PC market.

Another surprise was the cost of the disk arrays. With redundant power supplies and fans, the ability to “hot-swap” a disk drive, the RAID hardware controller itself, the redundant disks, and so on, the first disk arrays cost many times the cost of the disks. Perhaps as a result, the “inexpensive” in RAID morphed into “independent.” Many marketing departments and technical writers today know of RAID only as “redundant arrays of independent disks.”

In 2004, more than 80% of the nondesktop drive sales were found in RAIDs. In recognition of their role, in 1999 Garth Gibson, Randy Katz, and David Patterson received the IEEE Reynold B. Johnson Information Storage Award “for the development of Redundant Arrays of Inexpensive Disks (RAID).”

Protection Mechanisms

Architectural support for protection has varied greatly over the past 20 years. In early computers, before virtual memory, protection was very simple at best. In the 1960s, more sophisticated mechanisms that supported different protection levels (called *rings*) were invented. In the late 1970s and early 1980s, very elaborate mechanisms for protection were devised and later built; these mechanisms supported a variety of powerful protection schemes that allowed controlled instances of sharing, in such a way that a process could share data while controlling exactly what was done to the data. The most powerful method, called *capabilities*, created a data object that described the access rights to some portion of memory. These capabilities could then be passed to other processes, thus granting access to the object described by the capability. Supporting this sophisticated protection mechanism was both complex and costly, because creation, copying, and manipulation of capabilities required a combination of operating system and hardware support. Recent computers all support a simpler protection scheme based on virtual memory, similar to that discussed in [Section 5.7](#). Given current concerns about computer security due to the costs of worms and viruses, perhaps we will see a renaissance in protection research, potentially renewing interest in 20-year-old publications.

As mentioned in the text, system virtual machines were pioneered at IBM as part of its investigation into virtual memory. IBM’s first computer with virtual memory was the IBM 360/67, introduced in 1967. IBM researchers wrote the program CP-67, which created the illusion of several independent 360 computers. They then wrote an interactive, single-user operating system called CMS that ran on these virtual machines. CP-67 led to the product VM/370, and today IBM sells z/VM for its mainframe computers.

A Brief History of Modern Operating Systems

MIT developed the first timesharing system, CTSS (Compatible Time-Sharing System), in 1961. John McCarthy is generally given credit for the idea of timesharing, but Fernando Corbato was the systems person who realized the concept in the form of the CTSS. CTSS allowed three people to share a machine, and its response time of minutes or seconds was a dramatic improvement over the batch processing system it replaced. Moreover, it demonstrated the value of interactive computing.

Flush with the success of their first system, this group launched into their second system, MULTICS (Multiplexed Information and Computing Service). They included many innovations, such as strong protection, controlled sharing, and dynamic libraries. However, it suffered from the “second system effect.” Fred Brooks, Jr. described the second system effect in his classic book about lessons learned from developing an operating system for the IBM mainframe, *The Mythical Man Month*:

When one is designing the successor to a relatively small, elegant, and successful system, there is a tendency to become grandiose in one's success and design an elephantine feature-laden monstrosity.

MULTICS took sharing to a logical extreme to discover the issues, including that it was too extreme. MIT, General Electric, and later Bell Labs all tried to build an economical and useful system. Despite a great deal of time and money, they failed.

UC Berkeley was building its own timesharing system, Cal TSS. (“Cal” is a nickname for University of California.) The people leading that project included Peter Deutsch, Butler Lampson, Chuck Thacker, and Ken Thompson. They added paging virtual memory hardware to an SDS 920 and wrote an operating system for it. SDS sold this computer as the SDS-930, and it was the first commercially available timesharing system to have operational hardware and software. Thompson graduated and joined Bell Labs. The others founded Berkeley Computer Corporation (BCC), with the goal of selling time-sharing hardware and software. We'll pick up BCC later in the story, but for now let's follow Thompson.

At Bell Labs in 1971, Thompson led the development of a simple timesharing system that had some of the good ideas of MULTICS but left out many of the complex features. To demonstrate the contrast, it was first called UNICS. As they were joined by others at Bell Labs who had been burned from the MULTICS experience, it was renamed UNIX, with the *x* coming from Phoenix, the legendary bird that rose from the ashes.

Their result was the most elegant operating system ever built. Forced to live in the 16-bit address space of the DEC minicomputers, it had an amazing amount of functionality per line of code. Major contributions were pipes, a uniform file system, a uniform process model, and the shell user interface that allowed users to connect programs together using pipes and files.

Dennis Ritchie joined the UNIX team in 1973 from MIT, where he had experience in MULTICS, which was written in a high-level language. Like prior operating systems, UNIX had been written in assembly language. Ritchie designed a language for system implementation called C, and it was used to make UNIX portable.

Between 1971 and 1976, Bell released six editions of the UNIX timesharing system. Thompson took a sabbatical at his alma mater and brought UNIX with him. Berkeley and many other universities began to use UNIX on the popular PDP-11 minicomputer.

When DEC announced the VAX, a 32-bit virtual address successor to the PDP-11, the question arose as to what operating system should be run. UNIX became the first operating system to be migrated to a different computer when it was ported to the VAX.

Students at Berkeley had one of the first VAXes, and they were soon adding features to UNIX for the VAX, such as paging and a very efficient implementation of the TCP/IP protocol. The Berkeley implementation of TCP/IP was notable not just because it was fast. It was essentially the *only* implementation of TCP/IP for years, since early implementations in most other operating systems consisted of copying the Berkeley code verbatim, with minimal changes to integrate into the local system.

The Advanced Research Project Agency (ARPA), which funded computer science research, asked a Stanford professor, Forrest Baskett, to recommend which system the academic community should use: the DEC operating system VMS, led by David Cutler, or the Berkeley version of UNIX, led by a graduate student named Bill Joy. He recommended the latter, and Berkeley UNIX soon became the academic standard bearer.

The Berkeley Software Distribution (BSD) of UNIX, first released in 1978, was essentially one of the first open source movements. The sources were shipped with the tapes, and systems developers around the world learned their craft by studying the UNIX code.

BSD was also the first split of UNIX, because AT&T Bell Labs continued to develop UNIX on its own. This eventually led to a forest of UNIXes, as each company compiled the UNIX source code for their architecture. Bill Joy graduated from Berkeley and helped found Sun Microsystems, so naturally Sun OS was based on BSD UNIX. Among the many UNIX flavors were Santa Cruz Operation UNIX, HP-UX, and IBM's AIX. AT&T and Sun attempted to unify UNIX by striking a deal whereby AT&T and Sun would combine forces and jointly develop AT&T UNIX. This led to an adverse reaction from HP, IBM, and others, because they did not want a competitor supplying their code, so they created the Open Source Foundation as a competing organization.

In addition to the UNIX variants from companies, public domain versions also proliferated. The BSD team at Berkeley rewrote substantial portions of UNIX so that they could distribute it without needing a license from AT&T. This eventually led to a lawsuit, which Berkeley won. BSD UNIX soon split into FreeBSD, NetBSD, and OpenBSD, provided by competing camps of developers. Apple's current operating system, OS X, is based on Free BSD.

Let's go back to Berkeley Computer Corporation. Alas, this effort was not commercially viable. About the same time as BCC was getting in trouble, Xerox hired Robert Taylor to build the computer science division of the new Xerox Palo Alto

Research Center (PARC) in 1970. He had just returned from a tour of duty at ARPA, where he had funded the Berkeley research. He recruited Deutsch, Lampson, and Thacker from BCC to form the core of PARC's team: 11 of the initial 20 employees were from BCC, and they decided to build small computers for individuals rather than large computers for groups. This first personal computer, called the Alto, was built from the same technology as minicomputers, but it had a keyboard, mouse, graphical display, and windows. It popularized windows and led to many inventions, including client-server computing, the Ethernet, and print servers. It directly inspired the Macintosh, which was the successor to the popular Apple II.

IBM had long been interested in selling to the home, so the success of the Apple II led IBM to start a competing project. In contrast to its tradition, for this project IBM designed everything from components outside of the company. They selected the new 16-bit microprocessor from Intel, the 8086. (To lower costs, they started with the version with the 8-bit bus, called the 8088.) They visited Microsoft to see if this small company would be willing to sell their popular Basic interpreter and asked for recommendations for an operating system. Gates volunteered that Microsoft could deliver both an interpreter and an operating system, as long as they were paid a royalty fee of between \$10 and \$50 for each copy rather than a flat fee. IBM agreed, provided Microsoft could meet their deadlines. Microsoft didn't have an operating system, nor the time and resources to build one, but Gates knew that a Seattle company had developed an operating system for the Intel 8086. Microsoft purchased QDOS (Quick and Dirty Operating System) for \$15,000, made a small change and relabeled it MS-DOS. MS-DOS was a simple operating system without any modern features—no protection, no processes, and no virtual memory—in part because they believed it wasn't necessary for a personal computer.

Announced in 1980, the IBM PC became a tremendous success for IBM and the companies it relied upon. Microsoft sold 500,000 copies of MS-DOS by 1983, and the \$10 million income allowed Microsoft to start new software projects.

After seeing a version of the Macintosh under development, Microsoft hired some people from PARC to lead its reply. The Macintosh was announced in 1984, and Windows was available on PCs the following year. It was originally an application that ran on top of DOS, but was later integrated with DOS and renamed Windows 2.0. Microsoft hired Cutler from DEC to lead the development of Windows NT, a new operating system. NT was a modern operating system with protection, processors, and so on and has much in common with DEC's VMS. Today's PC operating systems are more sophisticated than any of the timesharing systems of 20 years ago, yet they still suffer from the need to maintain compatibility with the crippled first PC operating systems such as MS-DOS.

The popularity of the PC led to a desire for a UNIX that ran on it. Many tried to develop one, but the most successful was written from scratch in 1991 by Linus Torvalds. In addition to making the source code available, like BSD, he allowed everyone to make changes and submit them for inclusion in his next release. Linux popularized open source development as we know it today, with such software getting hundreds of volunteers to test releases and add new features.

Many people in this story won awards for their roles in the development of modern operating systems. McCarthy received an ACM Turing Award in 1971 in part for his contributions to timesharing. In 1983, Thompson and Ritchie received one for UNIX. The announcement said that “the genius of the UNIX system is its framework, which enables programmers to stand on the work of others.” In 1990, Corbato received the Turing Award for his contributions to CTSS and MULTICS. Two years later, Lampson won it in part for his work on personal computing and operating systems.

Further Reading

Brooks, F. P. [1975]. *The mythical man-month*. Reading: Addison-Wesley.

The classic book that explains the challenge of software engineering using IBM OS development as the example.

Cantin, J. F. and M. D. Hill [2001]. “Cache performance for selected SPEC CPU2000 benchmarks”, *SIGARCH Computer Architecture News* 29:4 ♪ (September), 13–18.

A reference paper of cache miss rates for many cache sizes for the SPEC2000 benchmarks.

Chen, P. M., E. K. Lee, G. A. Gibson, R. H. Katz, and D. A. Patterson [1994]. “RAID: High-performance, reliable secondary storage”, *ACM Computing Surveys* 26:2 (June) 145–88.

A tutorial covering disk arrays and the advantages of such an organization.

Conti, C., D. H. Gibson, and S. H. Pitowsky [1968]. “Structural aspects of the System/360 Model 85, part I: General organization”, *IBM Systems J.* 7:1, 2–14.

A classic paper that describes the first commercial computer to use a cache and its resulting performance.

Hennessy, J. and D. Patterson [2003]. [Chapter 5](#) in *Computer Architecture: A Quantitative Approach*, third edition, Morgan Kaufmann Publishers, San Francisco.

For more in-depth coverage of a variety of topics including protection, cache performance of out-of-order processors, virtually addressed caches, multilevel caches, compiler optimizations, additional latency tolerance mechanisms, and cache coherency.

Kilburn, T., D. B. G. Edwards, M. J. Lanigan, and F. H. Sumner [1962]. “One-level storage system”, *IRE Transactions on Electronic Computers* EC-11 (April), 223–335. Also appears in D. P. Siewiorek, C. G. Bell, and A. Newell [1982], *Computer Structures: Principles and Examples*, McGraw-Hill, New York, 135–48.

This classic paper is the first proposal for virtual memory.

LaMarca, A. and R. E. Ladner [1996]. “The influence of caches on the performance of heaps”, *ACM J. of Experimental Algorithmics*, Vol. 1.

This paper shows the difference between complexity analysis of an algorithm, instruction count performance, and memory hierarchy for four sorting algorithms.

McCalpin, J.D. [1995]. “STREAM: Sustainable Memory Bandwidth in High Performance Computers”, <https://www.cs.virginia.edu/stream/>.

A widely used microbenchmark that measures the performance of the memory system behind the caches.

Patterson, D., G. Gibson, and R. Katz [1988]. “A case for redundant arrays of inexpensive disks (RAID)”, *SIGMOD Conference*, 109–116.

A classic paper that advocates arrays of smaller disks and introduces RAID levels.

Przybylski, S. A. [1990]. *Cache and Memory Hierarchy Design: A Performance-Directed Approach*, Morgan Kaufmann Publishers, San Francisco.

A thorough exploration of multilevel memory hierarchies and their performance.

Ritchie, D. [1984]. "The evolution of the UNIX time-sharing system", *AT&T Bell Laboratories Technical Journal* 1984, 1577–1593.

The history of UNIX from one of its inventors.

Ritchie, D. M. and K. Thompson [1978]. "The UNIX time-sharing system", *Bell System Technical Journal* (August), 1991–2019.

A paper describing the most elegant operating system ever invented.

Silberschatz, A., P. Galvin, and G. Grange [2003]. *Operating System Concepts*, sixth edition, Addison-Wesley, Reading, MA.

An operating systems textbook with a thorough discussion of virtual memory, processes and process management, and protection issues.

Smith, A. J. [1982]. "Cache memories," *Computing Surveys* 14:3 (September), 473–530.

The classic survey paper on caches. This paper defined the terminology for the field and has served as a reference for many computer designers.

Smith, D. K. and R. C. Alexander [1988]. *Fumbling the Future: How Xerox Invented, Then Ignored, the First Personal Computer*, Morrow, New York.

A popular book that explains the role of Xerox PARC in laying the foundation for today's computing, but which Xerox did not substantially benefit from.

Tanenbaum, A. [2001]. *Modern Operating Systems*, second edition, Upper Saddle River, Prentice Hall, NJ.

An operating system textbook with a good discussion of virtual memory.

Wilkes, M. [1965]. "Slave memories and dynamic storage allocation", *IEEE Trans. Electronic Computers* EC-14:2 (April), 270–71.

The first classic paper on caches.

5.1.6 [15] <§5.1> How many 16-byte cache blocks are needed to store all 64-bit matrix elements being referenced using Matlab's matrix storage? How many using C's matrix storage? (Assume each row contains more than one element.)

5.2 Caches are important to providing a high-performance memory hierarchy to processors. Below is a list of 64-bit memory address references, given as word addresses.

0x03, 0xb4, 0x2b, 0x02, 0xbf, 0x58, 0xbe, 0x0e, 0xb5,
0x2c, 0xba, 0xfd

5.2.1 [10] <§5.3> For each of these references, identify the binary word address, the tag, and the index given a direct-mapped cache with 16 one-word blocks. Also list whether each reference is a hit or a miss, assuming the cache is initially empty.

5.2.2 [10] <§5.3> For each of these references, identify the binary word address, the tag, the index, and the offset given a direct-mapped cache with two-word blocks and a total size of eight blocks. Also list if each reference is a hit or a miss, assuming the cache is initially empty.

5.2.3 [20] <§§5.3, 5.4> You are asked to optimize a cache design for the given references. There are three direct-mapped cache designs possible, all with a total of eight words of data:

- C1 has 1-word blocks,
- C2 has 2-word blocks, and
- C3 has 4-word blocks.

5.3 By convention, a cache is named according to the amount of data it contains (i.e., a 4 KiB cache can hold 4 KiB of data); however, caches also require SRAM to store metadata such as tags and valid bits. For this exercise, you will examine how a cache's configuration affects the total amount of SRAM needed to implement it as well as the performance of the cache. For all parts, assume that the caches are byte addressable, and that addresses and words are 64 bits.

5.3.1 [10] <§5.3> Calculate the total number of bits required to implement a 32 KiB cache with two-word blocks.

5.3.2 [10] <§5.3> Calculate the total number of bits required to implement a 64 KiB cache with 16-word blocks. How much bigger is this cache than the 32 KiB cache described in Exercise 5.3.1? (Notice that, by changing the block size, we doubled the amount of data without doubling the total size of the cache.)

5.3.3 [5] <§5.3> Explain why this 64 KiB cache, despite its larger data size, might provide slower performance than the first cache.

5.3.4 [10] <§5.3, 5.4> Generate a series of read requests that have a lower miss rate on a 32 KiB two-way set associative cache than on the cache described in Exercise 5.3.1.

5.4 [15] <§5.3> Section 5.3 shows the typical method to index a direct-mapped cache, specifically (Block address) modulo (Number of blocks in the cache). Assuming a 64-bit address and 1024 blocks in the cache, consider a different indexing function, specifically (Block address[63:54] XOR Block address[53:44]). Is it possible to use this to index a direct-mapped cache? If so, explain why and discuss any changes that might need to be made to the cache. If it is not possible, explain why.

5.5 For a direct-mapped cache design with a 64-bit address, the following bits of the address are used to access the cache.

Tag	Index	Offset
63–10	9–5	4–0

5.5.1 [5] <§5.3> What is the cache block size (in words)?

5.5.2 [5] <§5.3> How many blocks does the cache have?

5.5.3 [5] <§5.3> What is the ratio between total bits required for such a cache implementation over the data storage bits?

Beginning from power on, the following byte-addressed cache references are recorded.

Address												
Hex	00	04	10	84	E8	A0	400	1E	8C	C1C	B4	884
Dec	0	4	16	132	232	160	1024	30	140	3100	180	2180

5.5.4 [20] <§5.3> For each reference, list (1) its tag, index, and offset, (2) whether it is a hit or a miss, and (3) which bytes were replaced (if any).

5.5.5 [5] <§5.3> What is the hit ratio?

5.5.6 [5] <§5.3> List the final state of the cache, with each valid entry represented as a record of <index, tag, data>. For example,

<0, 3, Mem[0xC00]–Mem[0xC1F]>

5.6 Recall that we have two write policies and two write allocation policies, and their combinations can be implemented either in L1 or L2 cache. Assume the following choices for L1 and L2 caches:

L1	L2
Write through, non-write allocate	Write back, write allocate

5.6.1 [5] <§§5.3, 5.8> Buffers are employed between different levels of memory hierarchy to reduce access latency. For this given configuration, list the possible buffers needed between L1 and L2 caches, as well as L2 cache and memory.

5.6.2 [20] <§§5.3, 5.8> Describe the procedure of handling an L1 write-miss, considering the components involved and the possibility of replacing a dirty block.

5.6.3 [20] <§§5.3, 5.8> For a multilevel exclusive cache configuration (a block can only reside in one of the L1 and L2 caches), describe the procedures of handling an L1 write-miss and an L1 read-miss, considering the components involved and the possibility of replacing a dirty block.

5.7 Consider the following program and cache behaviors.

Data Reads per 1000 Instructions	Data Writes per 1000 Instructions	Instruction Cache Miss Rate	Data Cache Miss Rate	Block Size (bytes)
250	100	0.30%	2%	64

5.7.1 [10] <§§5.3, 5.8> Suppose a CPU with a write-through, write-allocate cache achieves a CPI of 2. What are the read and write bandwidths (measured by bytes per cycle) between RAM and the cache? (Assume each miss generates a request for one block.)

5.7.2 [10] <§§5.3, 5.8> For a write-back, write-allocate cache, assuming 30% of replaced data cache blocks are dirty, what are the read and write bandwidths needed for a CPI of 2?

5.8 Media applications that play audio or video files are part of a class of workloads called “streaming” workloads (i.e., they bring in large amounts of data but do not reuse much of it). Consider a video streaming workload that accesses a 512 KiB working set sequentially with the following word address stream:

0, 1, 2, 3, 4, 5, 6, 7, 8, 9 . . .

5.8.1 [10] <§§5.4, 5.8> Assume a 64 KiB direct-mapped cache with a 32-byte block. What is the miss rate for the address stream above? How is this miss rate sensitive to the size of the cache or the working set? How would you categorize the misses this workload is experiencing, based on the 3C model?

5.8.2 [5] <§§5.1, 5.8> Re-compute the miss rate when the cache block size is 16 bytes, 64 bytes, and 128 bytes. What kind of locality is this workload exploiting?

5.8.3 [10] <§5.13> “Prefetching” is a technique that leverages predictable address patterns to speculatively bring in additional cache blocks when a particular cache block is accessed. One example of prefetching is a stream buffer that prefetches sequentially adjacent cache blocks into a separate buffer when a particular cache block is brought in. If the data are found in the prefetch buffer, it is considered